GAS FLOW SENSOR CIRCUIT DEVELOPMENT & DATA ACQUISITION

SUMMER TRAINING PROJECT REPORT

BY

AKASH LAHA

(T91/IE/224116)

RAJABAZAR SCIENCE COLLEGE

(UNIVERSITY OF CALCUTTA)

DEPARTMENT OF

VARIABLE ENERGY CYCLOTRON CENTRE, KOLKATA

2nd JUNE – 15th JULY, 2025

VARIABLE ENERGY CYCLOTRON CENTRE

1/AF, Bidhannagar, Sector-1, Kolkata-700064

BONAFIDE CERTIFICATE

This is to certify that this project report entitled “***GAS FLOW SENSOR CIRCUIT DEVELOPMENT & DATA ACQUISITION***” submitted to Variable Energy Cyclotron Centre, Kolkata, is a record of work done by “AKASH LAHA”, Roll No. T91/IE/224116, 3rd year B.Tech student of RAJABAZAR SCIENCE COLLEGE, KOLKATA, under my supervision from “2nd June 2025” to “15th July 2025”.

Jogender Saini

Scientific Officer, VECC

(Project Guide)

Place: VECC, Kolkata

Date:

DECLARATION BY AUTHOR

This is to declare that this report has been written by me. No part of the report is plagiarized from other sources. All information included from other sources has been duly acknowledged. I declare that if any part of the report is found to be plagiarized, I shall take full responsibility for it.

AKASH LAHA

IE, 3rd Year

UNIVERSITY OF CALCUTTA

Roll number: T91/IE/224116

Place: Kolkata

Date:

<u>Acknowledgment</u>

I sincerely thank **Variable Energy Cyclotron Centre (VECC), Kolkata**, for giving me the opportunity to undertake a one-month internship. I express my heartfelt gratitude to my project guide **Mr Jogender Saini, Scientific Officer** and coordinator **Ms Anamika Pallavi, Scientific Assistant** at VECC, for their valuable support and mentorship throughout the internship period. I also extend my sincere thanks to **Dr. Ranjini Menon, Scientific Officer** and **Tanmoy Das, Scientific Assistant** at (**VECC**).

Special thanks to our Head of the Department **Dr. Chanchal Kumar Dey** and faculty members **Dr. Ujjwal Mondal, Dr. Madhuchhanda Mitra and Dr. Sourav Pal** at our department of **Instrumentation Engineering** at **Rajabazar Science College, University of Calcutta** for their constant encouragement and for facilitating this internship.

First and foremost, I extend my heartfelt thanks to the **Electronics Engineering Division at VECC**, where I was fortunate to work. Their guidance, encouragement, and constant support throughout the internship period made it possible for me to gain hands-on experience in **analog circuit development, PCB designing & FPGA-based systems**. I am deeply grateful to my mentor and coordinators who generously shared their knowledge and expertise, patiently answering my questions and helping me understand complex concepts.

I also appreciate the warm and welcoming environment at **VECC**, which fostered learning and innovation. The access to advanced instrumentation, cutting-edge technologies, and research facilities provided me with practical exposure that textbooks alone cannot offer. The collaborative spirit and professional culture at **VECC** motivated me to apply theoretical knowledge into real-world applications and enhanced my problem-solving skills.

This internship has not only deepened my understanding of digital systems and embedded design but also inspired me to pursue further research and development in this field. I am confident that the skills and experience gained during this period will serve as a strong foundation for my future career.

Finally, I thank my family and friends for their encouragement and support throughout this journey.

I look forward to maintaining a continued association with **VECC** and hope to contribute to its future endeavours in scientific research and technological advancements.

Table of Contents

CHAPTER-1

INTRODUCTION OF VECC..**6**

PROJECT DESCRIPTION 13

CHAPTER-2: PHYSICAL DESIGN

THERMAL TYPE GAS FLOW SENSOR 15

CONSTANT CURRENT SOURCE DESIGN...17

SIGNAL CONDITIONING 19

PCB DESIGN 22

CHAPTER-3: DATA ACQUISITION SYSTEM

TOOLS & TECHNOLOGIES USED 25

INTRODUCTION TO FPGA...25

INTRODUCTION TO VIVADO 28

INTRODUCTION TO VHDL...30

COMMUNICATION SETUP 33

UART COMMUNICATION...33

DAQ SYSTEM VIA ADCs...39

DATA FLOW DIAGRAM..43

RESULT AND ANALYSIS...43

CONCLUSION...47

REFERENCES 48

CHAPTER-1

Introduction to VECC

About VECC:

A premier research institute under the Department of Atomic Energy, Govt. of India, working in nuclear science and instrumentation.

The importance of Cyclotron for research in nuclear physics was conceived by Dr. Homi J. Bhabha in 1965 and the K130 machine similar to LBL K130 machine was decided upon. The activity at Kolkata (the then Calcutta) started in 1969 by BARC. The machine was commissioned in 1977 and first beam of alpha particle came out on 16th June, 1997.

The mandates of the Centre were to carry out research in experimental nuclear physics, radiation damage studies and isotope production for research and nuclear medicine. The centre became a national facility with full-fledged design and development facilities for mechanical engineering, power electronics, RF engineering and computation. To support the experimental nuclear physicist a strong theory group also emerged. To commemorate the significant event of the extraction of the first beam of alpha particle, VECC celebrates 16th June as its Foundation Day. The centre became an independent R&D unit of DAE in May, 1990.

In order to carry out nuclear physics in the domain of exotic nucleus a Radioactive Ion Beam facility has been built which would act as a part of the more complex accelerator facility under the major upcoming project named Advance National facility for Unstable and Rare Ion Beam (ANURIB). The RIB facility is presently being utilized for study of defect propagation and surface physics of materials.

At present VECC has 3 operating cyclotrons. K130 Cyclotron has been operating since 1977, K500 Superconducting Cyclotron has been delivering heavy-ion beams to the users for basic science research and 30 MeV Medical Cyclotron has been operating for the production and delivery of radiopharmaceuticals to the hospitals for cancer diagnostics, as a societal services to the nation by the DAE.

Mission and Vision:

The primary mission of VECC is to contribute to India’s progress in fundamental and applied nuclear science by advancing the development of particle accelerators and using them for cutting-edge research. The centre supports both **basic and applied research**, providing infrastructure for Indian and international scientists to conduct experiments in nuclear and particle physics.

VECC aims to be at the forefront of:

- Accelerator-based science and technology.
- Development of radiation detectors and instrumentation.
- Research in nuclear, particle, and accelerator physics.
- National and international scientific collaborations.

Major Facilities and Research Areas:

VECC is home to several state-of-the-art facilities that support advanced research and development:

1. K130 Cyclotron

One of VECC's landmark achievements is the commissioning of the **K130 Variable Energy Cyclotron**, which was among the first of its kind in India. It is capable of accelerating protons, deuterons, alpha particles, and heavy ions. It has been widely used for nuclear reaction studies, medical isotope production, and other nuclear experiments.

2. Superconducting Cyclotron (SCC)

VECC also houses a **Superconducting Cyclotron**, a more advanced accelerator designed to provide beams of higher energy and intensity. It is used for experiments in heavy-ion physics, nuclear structure studies, and testing materials under extreme radiation.

3. Beam Transport Systems

VECC maintains complex beam transport and control systems to guide and utilize the accelerated particles effectively across multiple research areas.

4. Radiation Monitoring and Instrumentation

A major focus area at VECC is the development of **radiation detection systems**, monitoring equipment, and digital instrumentation essential for nuclear facilities and scientific experiments.

5. Computing and Data Acquisition

VECC has advanced computing facilities for simulation, data acquisition (DAQ), and analysis. These systems are crucial for experiments involving large datasets, especially in high-energy physics.

Electronics and Communication Division:

As part of its extensive scientific infrastructure, VECC has a highly specialized **Electronics and Communication Engineering Division**, where engineers and researchers design, develop, and maintain sophisticated electronic systems. These include:

- Embedded systems and FPGA-based solutions
- Digital communication circuits
- High-speed data acquisition boards

- Control systems for accelerator operations
- Sensor interfaces and signal conditioning circuits

Interns in this division gain exposure to tools such as **VHDL/Verilog**, **Xilinx Vivado**, **MATLAB**, **LabVIEW**, and **microcontroller programming**, as well as real-world design methodologies in electronics and instrumentation.

Collaborations and Outreach:

VECC actively collaborates with other national and international institutions, including:

- **Bhabha Atomic Research Centre (BARC)**
- **Tata Institute of Fundamental Research (TIFR)**
- **Saha Institute of Nuclear Physics (SINP)**
- **Large Hadron Collider (LHC) at CERN**

Through these collaborations, VECC contributes to large-scale experiments in nuclear and particle physics and helps advance the global scientific community's understanding of fundamental forces and particles.

Training and Internship Opportunities:

VECC regularly offers **internship and training programs** for undergraduate and postgraduate students in physics, electronics, and engineering disciplines. These internships are structured to provide:

- Hands-on training in real-world research labs
- Experience with FPGA and embedded systems
- Exposure to high-energy physics and nuclear instrumentation
- Mentoring by scientists and experienced engineers

These programs are highly valued for the experience they provide in working with state-of-the-art equipment and being part of national-level scientific projects.

6. Department/Division Overview

The **Electronics and Communication Engineering (ECE) Division** is dedicated to the study, research, and development of electronic devices, circuits, communication systems, and signal processing technologies. This division plays a pivotal role in advancing modern communication infrastructure and electronic system design by integrating theoretical knowledge with practical implementation.

Nuclear and Particle Physics Research at VECC, Kolkata:

The **Variable Energy Cyclotron Centre (VECC)** in Kolkata is one of India's premier research institutions engaged in **nuclear and particle physics**, operating under the **Department of Atomic Energy (DAE)**. Since its establishment, VECC has significantly contributed to the understanding of atomic nuclei, fundamental particles, and their interactions. The centre serves as a major hub for experimental and theoretical research in these areas and provides facilities for scientists from across India and abroad.

Importance of Nuclear and Particle Physics:

Nuclear physics focuses on understanding the structure and behavior of atomic nuclei, the forces that hold them together, and the processes that govern nuclear reactions. It plays a crucial role in energy generation, medical imaging, radiation therapy, and national security.

Particle physics, also known as high-energy physics, studies the most fundamental constituents of matter and the forces through which they interact. This field aims to answer profound questions about the origin of mass, the nature of dark matter, and the unification of fundamental forces.

VECC contributes to both these fields by providing state-of-the-art facilities, accelerator-based experiments, and active participation in global collaborations.

Key Facilities Supporting Research at VECC:

1. K130 Variable Energy Cyclotron

The **K130 cyclotron** is one of VECC’s most significant assets, capable of accelerating ions and particles to high energies. Researchers use it to study:

- Nuclear reactions at different energy levels
- Fission and fusion processes
- Production of radioactive isotopes

These experiments help in understanding nuclear structure and reaction dynamics, and also have applications in medical and industrial fields.

2. Superconducting Cyclotron (SCC):

The **Superconducting Cyclotron** is a newer and more advanced facility. It is capable of accelerating heavier ions to higher energies with better precision and beam intensity. This enables:

- Exploration of exotic and unstable nuclei
- Investigation of heavy-ion collisions
- Studies of nuclear matter under extreme conditions (high temperature and pressure)

3. Accelerator Beamlines and Detector Systems:

VECC also has multiple experimental beamlines equipped with advanced detectors for:

- Gamma-ray spectroscopy
- Neutron detection
- Particle identification
- Time-of-flight measurements

These allow detailed analysis of the products of nuclear reactions and help test predictions from theoretical models.

Theoretical Research in Nuclear and Particle Physics:

VECC also has a strong theoretical physics group that works in parallel with experimental teams. Their research includes:

- **Nuclear structure calculations** using shell model and mean-field theories
- **Quantum chromodynamics (QCD)** simulations
- **Reaction mechanism modelling**
- **Computational nuclear physics** using high-performance computing

The interaction between theory and experiment strengthens the reliability of data interpretation and enhances the accuracy of scientific predictions.

International Collaborations in Particle Physics:

VECC is an active participant in several **international high-energy physics collaborations**, including:

- **CERN's ALICE experiment** at the Large Hadron Collider (LHC), studying quark-gluon plasma and the conditions of the early universe.
- **FAIR (Facility for Antiproton and Ion Research)** in Germany, where VECC contributes to detector development and data analysis.
- **J-PARC (Japan Proton Accelerator Research Complex)** and other global platforms.

These collaborations allow Indian scientists and engineers to contribute to global discoveries and bring back knowledge, expertise, and cutting-edge technology to VECC.

Applications and Broader Impact:

Nuclear and particle physics research at VECC not only advances scientific understanding but also contributes to:

- **Medical technologies** such as cancer therapy using proton and ion beams
- **Radiation detectors** for environmental and security monitoring
- **Nuclear data libraries** for reactor design and safety
- **Isotope production** for diagnostics and treatment

Furthermore, the research outputs feed into India's self-reliance in nuclear technology and high-precision instrumentation.

➢ **We have taken the internship in EHEPAG group in the electronics lab where we are working with the hands on experience on hardware development,FPGA programming and implementation .**

Experimental High Energy Physics and Applications Group(EHEPAG)

1. Research Focus

- Quark–Gluon Plasma (QGP) Studies: EHEPAG investigates QGP formation in heavy-ion collisions at ultra-relativistic energies. They contribute to both the STAR experiment at RHIC (Brookhaven) and the ALICE experiment at CERN (LHC)

---

2. Indigenous Detector Development

- Photon Multiplicity Detector (PMD): VECC led an Indian consortium (including IOP Bhubaneswar, IIT Bombay, Panjab Univ., Rajasthan Univ., Jammu Univ.) to design and build the PMD:
    - ~82,000 hexagonal proportional counters for STAR, operational since 2004
    - ~200,000 counters for ALICE, in use since 2009

3. Computing Infrastructure

- **ALICE Tier-2 Grid Centre**: VECC hosts a Tier-2 node, supporting ~2 % of ALICE's computing needs with >95 % uptime—key for data processing and storage.

4. Involvement in Global Experiments

- **India-based Neutrino Observatory (INO)**: EHEPAG develops Bakelite RPC detectors, contributing the prototype Iron Calorimeter (ICAL), which is commissioned and recording cosmic rays
- **Compressed Baryonic Matter (CBM) @ FAIR**: VECC is heavily involved in CBM, especially in building GEM-based muon detectors and associated electronics .

5. Detector Technology for Medical Imaging

- Experience from high-energy physics detector R&D is being applied to medical imaging:
  - Development & testing of MRPCs (glass and Bakelite)
  - Ongoing Monte Carlo simulations for their response to positron-annihilation photons.

6. Supporting Nuclear Physics Research

Besides high-energy work, VECC also contributes to mid-energy nuclear physics:

- The **INGA campaign** uses digital data acquisition systems and Clover-HPGe detectors for γ-ray spectroscopy.
- Detector arrays like **ChAKRA** (silicon + CsI(Tl) telescopes) support charged-particle spectroscopy .

Project Description

This report presents a comprehensive overview of the 45-day internship undertaken at the Variable Energy Cyclotron Centre (VECC), Kolkata, within the Electronics Engineering Division. The training program focused on the design and development of advanced hardware systems, incorporating **analog integrated circuits (ICs) and FPGA-based platforms**, with a strong emphasis on application-oriented, hands-on learning.

A major area of focus during the internship was the **Gas Electron Multiplier (GEM) detector**, a sophisticated particle detection system employed in high-energy physics experiments. The GEM detector operates by utilizing **gas ionization and subsequent electron multiplication to detect high-energy charged particles such as electrons and muons**. It features a thin insulating Kapton foil, copper-coated on both sides and densely perforated with microscopic holes. As charged particles traverse the gas-filled chamber, they ionize the gas molecules, releasing free electrons. These primary electrons drift toward the GEM foil under an applied electric field, and within the high-field regions of the holes, they undergo avalanche multiplication, producing thousands of secondary electrons. These multiplied electrons are then collected at the readout electrode, generating a detectable signal for precise particle tracking.

Maintaining a stable and contamination-free gas environment is critical to the optimal functioning of the GEM detector. In this context, the gas flow sensor plays a pivotal role. It ensures consistent and accurate flow of the gas mixture—**typically Argon and Carbon Dioxide (Ar–CO_2)—**which is vital for sustaining uniform gain and preventing impurities such as oxygen or moisture from degrading the detector's performance. The sensor actively **monitors the flow rate, detects anomalies such as leaks or blockages**, and can be integrated with automated control systems to trigger alarms or deactivate high voltage circuits under fault conditions. This ensures both operational reliability and the protection of sensitive detector components.

During the internship, the project team undertook the design and implementation of a thermal-type gas flow sensor system capable of **measuring Ar–CO_2 gas mixture flow rates in the range of 0–200 standard cubic centimeters per minute (sccm) for GEM detector applications**. The system development involved creating a custom constant current source to bias the sensor appropriately. Given that the sensor output was in the **nanovolt range and unsuitable for direct measurement, a high-precision analog amplifier circuit was designed to elevate the signal to a measurable level.**

To facilitate real-time monitoring and eliminate manual data logging, an Analog-to-Digital Converter (ADC) was integrated to digitize the amplified signal. The final data acquisition system was realized using FPGA hardware programmed in VHDL, with **a Python-based graphical user interface (GUI) developed to display real-time gas flow readings on a computer.**

This project provided in-depth exposure to practical circuit design, signal conditioning, analog and digital integration, and real-time embedded system development, thereby enhancing both theoretical understanding and applied engineering skills in the context of high-energy physics instrumentation.

CHAPTER-2: PHYSICAL DESIGN

Thermal type gas flow sensor

A sensor converts a physical quantity into a electrical signals .As the name mentioned above a flow sensor measures the flow rate of any fluid. In this project we have used a thermal type gas flow sensor which is built at semiconductor Laborotary (SCL), Chandigarh.It precisely measures the flow of gas in the range of 0-200 sccm(1-12 L/h).

Principle:

A thermal-type gas flow sensor works based on the principle of heat transfer from a heated element to a flowing gas. When a gas flows over a heated surface, it carries away some of the heat, and this heat loss is directly related to the flow rate of the gas. In sensors that use a **thermocouple and a heater**.A small resistive heater (usually made of metal or semiconductor) is placed in the path of the gas flow. This heater generates a known and controlled amount of heat.Two thermocouples are typically placed—one **upstream** and one **downstream** of the heater. These thermocouples measure the temperature of the gas before and after it passes over the heater.

Working:

When no gas is flowing, the heat generated by the heater is distributed symmetrically around it, and the upstream and downstream thermocouples measure similar temperatures. When gas starts to flow, it carries heat from the heater downstream. As a result the **downstream thermocouple** senses a higher temperature while the **upstream thermocouple** senses a lower temperature. The **temperature difference (ΔT)** between the upstream and downstream thermocouples increases with the flow rate of the gas.

he thermocouples generate voltage signals (Seebeck effect) corresponding to the temperatures they sense. This differential voltage is proportional to the flow rate and can be calibrated to measure gas flow in standard cubic centimeters per minute (sccm).

Configuration:

The thermal gas flow sensor used in this project features a 9-pin interface, of which only four pins are utilized for operation. The configuration is as follows:

- **Pin 4** serves as the input for a constant current source, typically set to **3- 3.5 mA** to power the internal heater.
- **Pin 8** is designated as the **ground** for the heater circuit.
- **Pins 2 and 6** provide the **differential output signals**, with **Pin 2** as the positive output and **Pin 6** as the negative output.

The sensor's internal heater has a nominal resistance of approximately **820 ohms**, which may fluctuate between **820 Ω and 860 Ω** depending on the gas flow conditions. This variation is a result of convective heat transfer as the gas flows over the heater.

Additionally, the sensor is equipped with dedicated **gas inlet and outlet ports** to facilitate controlled gas flow through the sensing chamber.

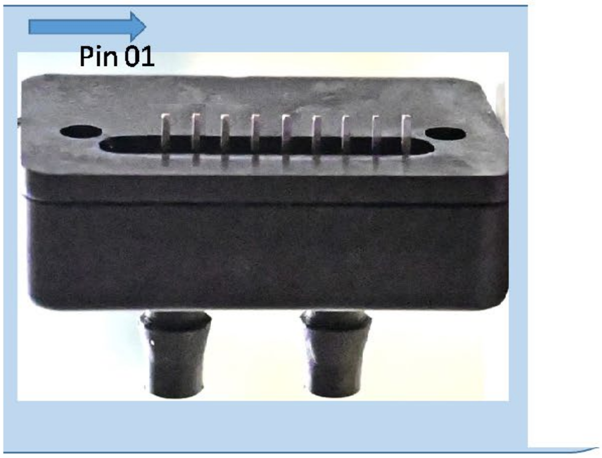


Figure.1

Figure.2

Constant current source design

As previously mentioned, the thermal gas flow sensor requires a **constant current supply of approximately 3.3 mA** for accurate operation. However, the available power supply for the overall circuit is limited to a dual-rail voltage of **±7V**. To meet this requirement, a dedicated constant current source was designed using the following key components:

1. ASM1117 Voltage Regulator

The **ASM1117** is a three-terminal low-dropout linear voltage regulator. In this design, it receives an input of **+7V** and provides a **regulated output of 3.3V**, which serves as the reference voltage for the current source circuit. This ensures a stable input for further conversion into a controlled current. Two capacitors each of 100nf is connected with both the pins 2 & 3 for filtering purpose.

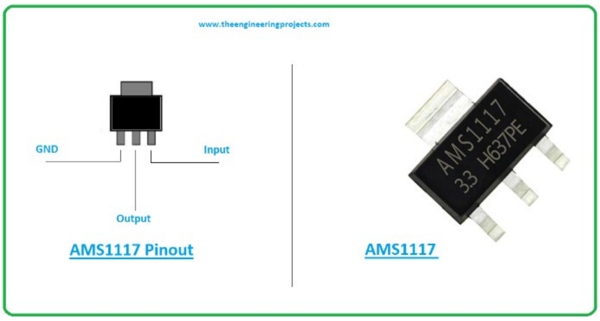


Figure.3 ASM1117

2. LM741 Operational Amplifier

The **LM741** is an 8-pin general-purpose operational amplifier (Op-Amp) used here in a voltage-to-current (V–I) converter configuration. It is powered by a **single supply of 0–7V**. A pair of **1 kΩ resistors** are used to set the gain of the amplifier to **2**, which helps in achieving the desired output characteristics .The V–I converter is designed such that the **output current through the load remains constant**, regardless of variations in the load resistance. The regulated 3.3V from the ASM1117 is used as the input to the Op-Amp circuit, which then drives current through the sensor heater. This approach ensures that the heater receives a steady current of approximately **3.3 mA**, which is crucial for the sensor's thermal stability and accurate performance.

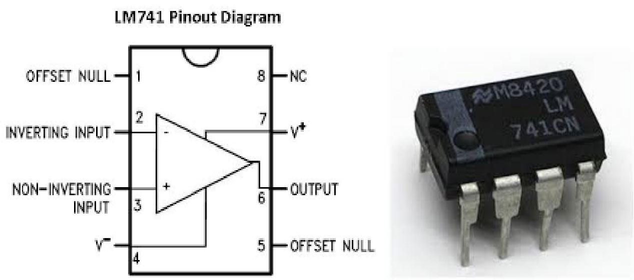


Figure.4 LM741

A detailed circuit diagram is provided below to illustrate the implementation of this constant current source in **figure 5.**

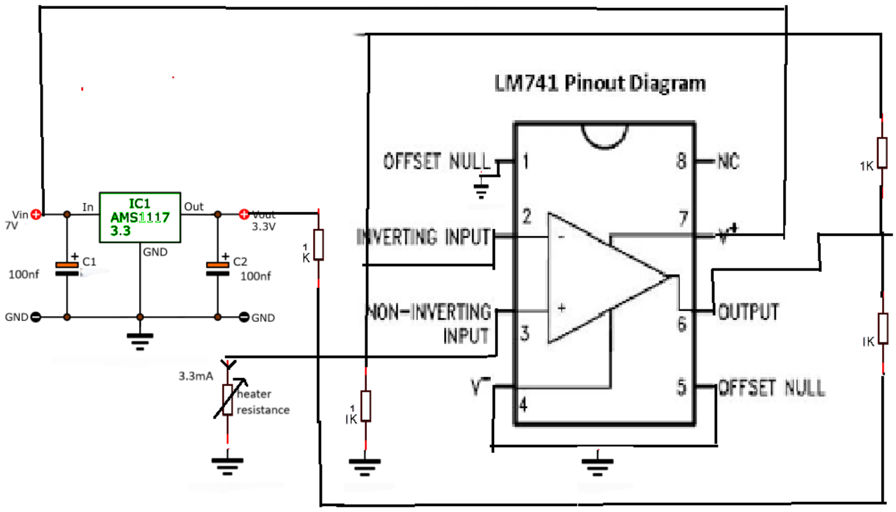


Figure.5 Constant current source of 3.3 mA

Signal Conditioning

The signal conditioning of the sensor output involves two main stages: amplification and analog-to-digital conversion. This process ensures that the weak sensor signal is converted into a format suitable for accurate digital processing.

- **Amplification Using AD8221:**

As the raw output from the sensor is too low to be directly processed, amplification is necessary to generate a usable signal. For this purpose, the AD8221 instrumentation amplifier was used due to its high input impedance, excellent common-mode rejection ratio (CMRR), and suitability for differential signal amplification.

The differential signal from pin 2 and pin 6 of the sensor was fed into the AD8221. The amplifier was operated with a dual power supply of ±7V, allowing it to handle signals centered around 0V. The reference pin (pin 5) of the AD8221 was connected to ground in order to set the output offset voltage to 0V, ensuring symmetric swing and improved performance.

To configure the desired gain, a 1 kΩ resistor was connected between pins 1 and 8 of the AD8221, resulting in a gain of approximately 50, as per the formula:

$$\mathbf{Gain = 1 + 49.4\ k\Omega / Rg}$$

This amplified signal is then passed on to the analog-to-digital converter for further processing.

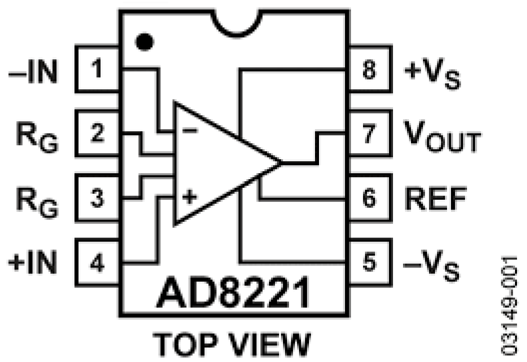


Figure.6 AD8221

- **Analog to Digital conversion Using AD7476:**

To convert the conditioned analog signal into a digital format suitable for further processing, the AD7476 analog-to-digital converter (ADC) was employed. The AD7476 is a high-speed, low-power, 12-bit ADC that operates with a supply voltage of 2.35V to 2.65V. In our design, a precise 2.5V reference voltage was required for stable operation, which was supplied using the VREF192 precision voltage reference IC. This ensured consistent performance and accuracy of the ADC throughout the data acquisition process.

The analog input to the AD7476 is sourced from the output of the instrumentation amplifier (AD8221), which amplifies the low-level differential signal from the sensor. This ensures that the ADC receives a properly scaled signal within its input range, thus maximizing resolution and accuracy.

The AD7476 communicates over a 3-wire serial interface, consisting of the Serial Clock (SCLK), Chip Select (CS), and Serial Data Output (SDATA) lines. In our setup, pins 5 (SDATA), 6 (SCLK), and 7 (CS) of the ADC are directly connected to the FPGA. This direct connection enables synchronized data transfer between the ADC and the FPGA, allowing high-speed, real-time digital signal acquisition.

Within the FPGA, a custom logic block was developed to handle serial data reception, synchronization, and conversion into a readable format for visualization. The digital data is then processed and displayed on a user interface, providing the user with real-time monitoring of the sensor output. This setup ensures a compact, efficient, and highly responsive data acquisition system.

Filtering must be done for ADCs (Analog-to-Digital Converters) to ensure accurate and reliable digital representation of the analog signal.It avoids aliasing, reduce noise, enhance accuracy, improve signal integrity and protect the ADC input.

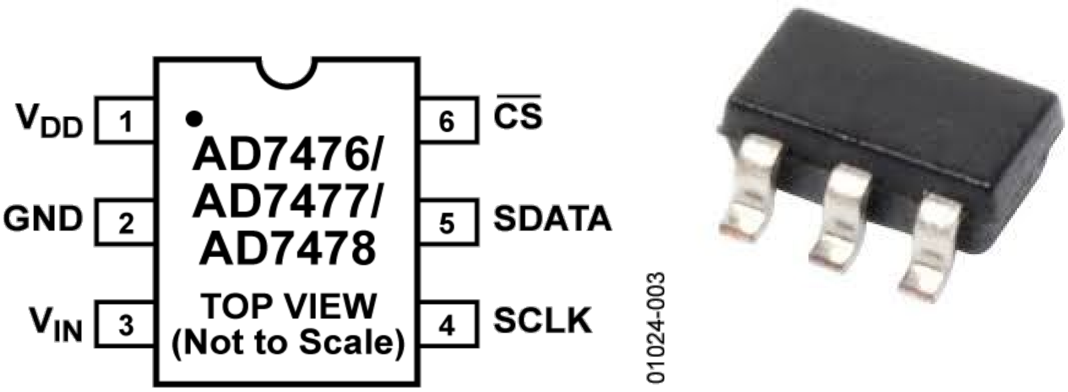


Figure.7

A circuit diagram is provided below to illustrate the signal conditioning of the sensor ouput signals in **figure.8.**

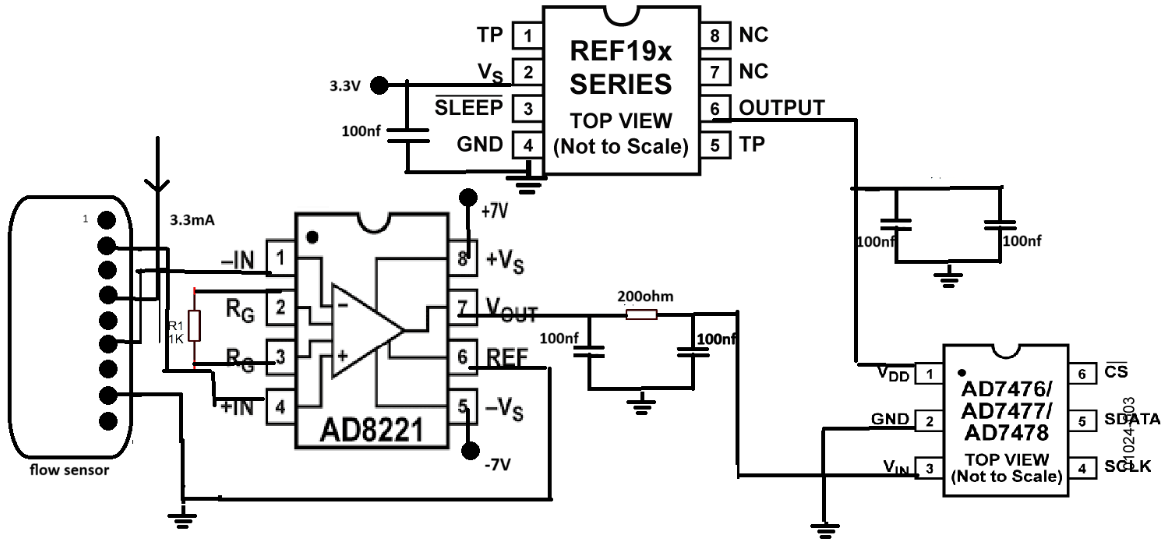


Figure.8 Signal conditioning of the output of flow sensor

Noise Cancellation:

After thorough testing of the complete circuit, it was observed that the output signal contained an undesirable level of noise, affecting the accuracy and reliability of the system. Effective noise mitigation is therefore critical, particularly in the power supply lines providing ±7V to sensitive analog components.

To address this, dedicated filter circuits have been integrated into each supply rail to suppress high-frequency noise and voltage fluctuations. These filters ensure a cleaner and more stable power delivery, which is essential for the proper functioning of analog front-end elements like the amplifier and ADC.

Furthermore, the increasing complexity of interconnections using discrete wiring not only introduces potential noise pickup but also makes the system prone to physical handling issues. To streamline the design and further minimize noise, a custom Printed Circuit Board (PCB) was developed. **The PCB layout was carefully designed with proper grounding, decoupling, and signal routing strategies to enhance noise immunity and ensure consistent, high-fidelity signal processing.**

Printed Circuit Board Design

To enhance the reliability, stability, and compactness of the circuit, a **Printed Circuit Board (PCB)** was designed using **Proteus Design Suite**. Unlike traditional breadboard or wired implementations, a PCB offers significant advantages in terms of reduced noise, improved signal integrity, and long-term durability.

Proteus provides an integrated environment for schematic capture and PCB layout, enabling precise component placement, efficient routing, and simulation capabilities. In this project, the schematic of the complete signal conditioning and data acquisition system was first developed, followed by the PCB layout design with careful attention to power supply decoupling, ground planes, and trace optimization to minimize electromagnetic interference and ensure a clean signal path. The finalized PCB layout was then used for fabrication and assembly, resulting in a robust and noise-resistant hardware platform.

- **PCB schematic & layout design:**

The schematic diagram and PCB layout of the complete circuit are illustrated in Figure 9 and Figure 10, respectively. The schematic captures all the essential components—including the signal conditioning stage, power supply filters, ADC interface, and necessary connectors—along with their interconnections. This schematic serves as the foundation for the PCB design process.

Following the schematic capture, the PCB layout was developed, emphasizing optimal component placement, minimal signal path lengths, and proper grounding to ensure signal integrity and minimize noise. Special care was taken to route analog and digital signals separately and to include decoupling capacitors near critical ICs.

In addition, a 3D visualization of the final PCB design is presented in Figure 11, offering a realistic preview of the physical board. This 3D view helps in verifying the mechanical fit, component orientation, and overall design compactness before actual fabrication.

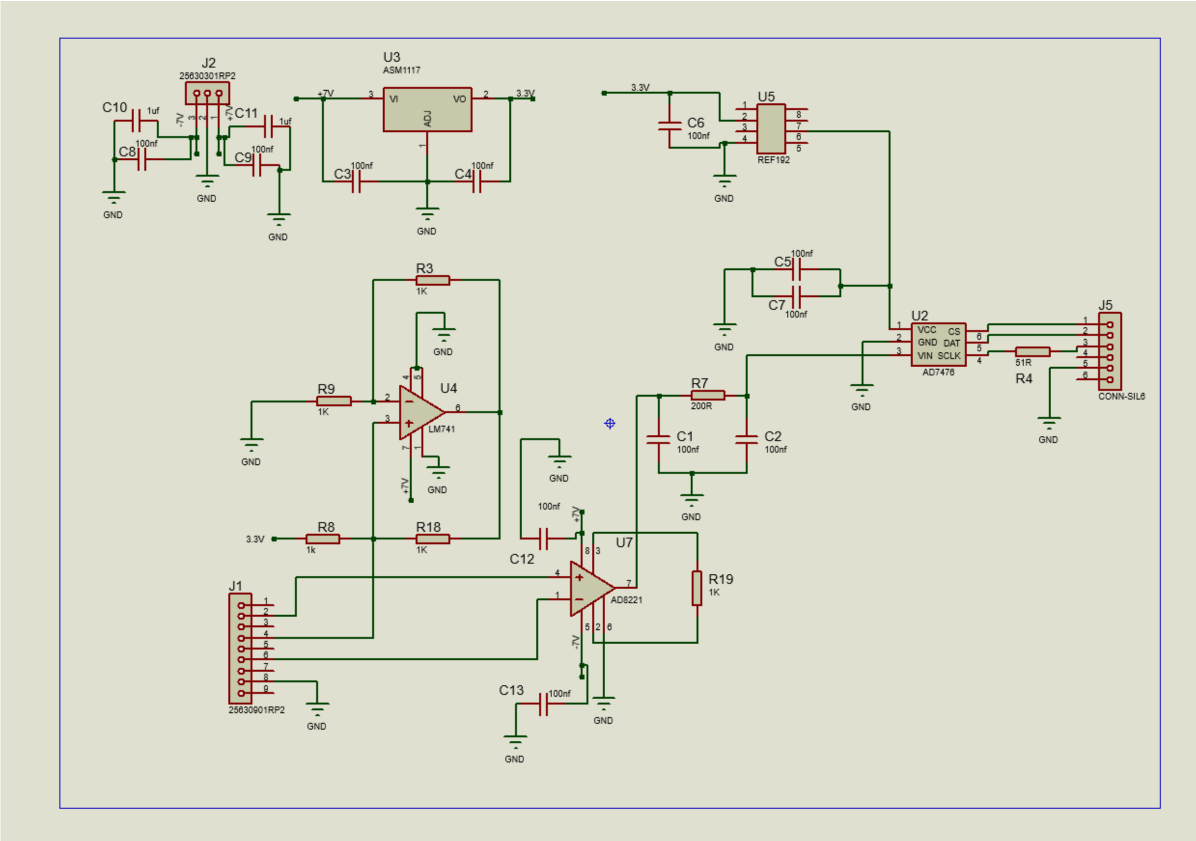


Figure.9 PCB schematic

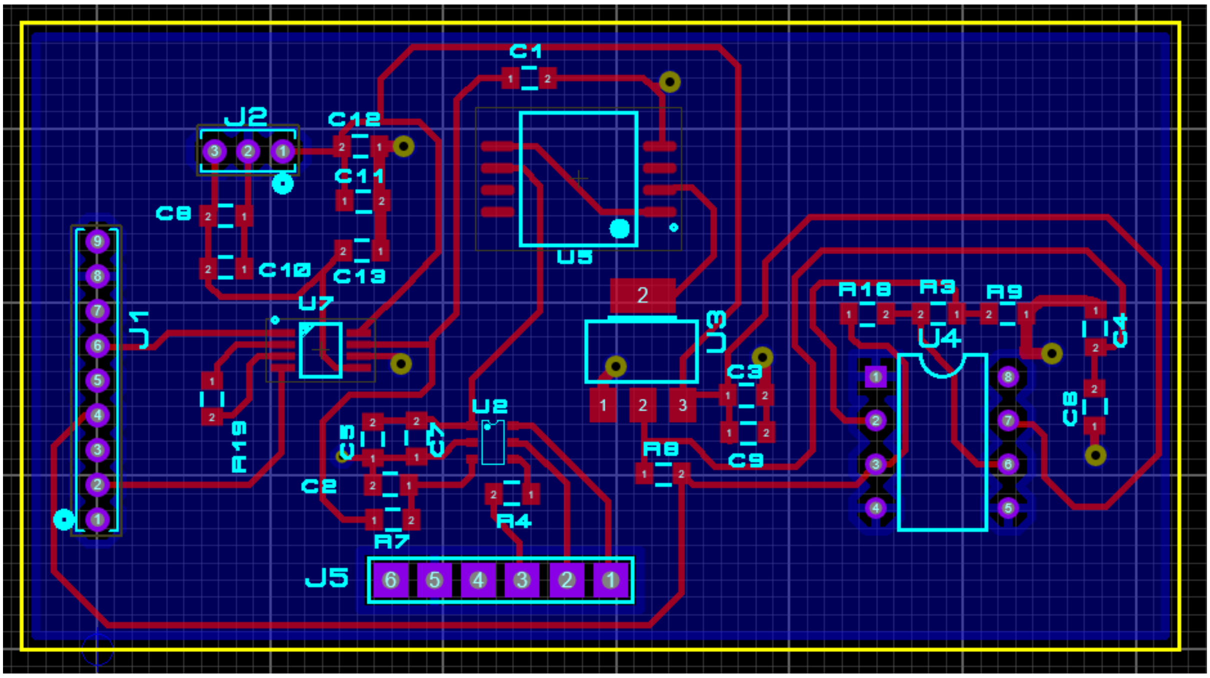


Figure.10 PCB layout

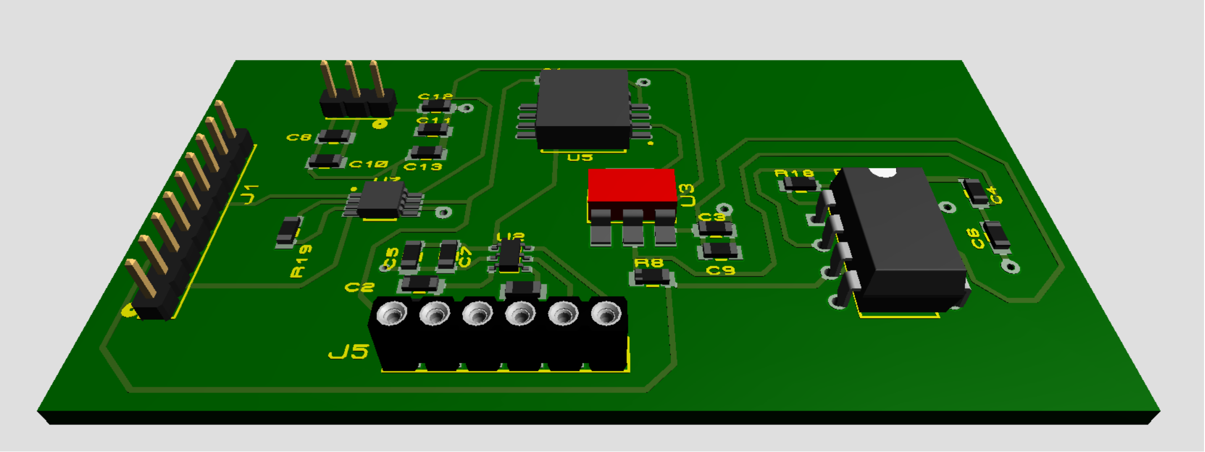


Figure.9 PCB schematic

CHAPTER-3: Data Acquisition System

A **Data Acquisition System (DAQ)** designed specifically for the gas flow measurement is essential for the precise and reliable capture, conversion, and analysis of gas flow rates in digital form. Accurate measurement of the flow rate of a gas mixture is critical for the experiment.

In this project, the DAQ system is implemented using a **Field Programmable Gate Array (FPGA)**, which offers a highly flexible and efficient platform for real-time data processing. The FPGA enables parallel processing, low latency, and deterministic control, making it well-suited for handling high-speed data acquisition tasks.

The data acquisition process begins with an **Analog-to-Digital Converter (ADC)** that converts the analog voltage signal output from the gas flow sensor into a digital format. This digitized data is then transmitted to the FPGA via **UART (Universal Asynchronous Receiver/Transmitter) communication protocol**, which facilitates serial data transfer between the ADC and FPGA in a reliable and synchronized manner.

The FPGA is programmed using **VHDL (VHSIC Hardware Description Language)**, a hardware description language that allows precise control over hardware behavior and timing. This enables efficient data reception, processing, and storage of the flow measurements for subsequent analysis.

Overall, this FPGA-based DAQ system provides a robust and scalable solution for accurate gas flow measurement, ensuring real-time data acquisition with high precision and reliability.

➢ Tools and Technologies Used:

- FPGA Board: Xilinx Artix-7
- Software: Xilinx Vivado (version 2023.2), Vivado Simulator
- Programming Language: VHDL
- Other: Oscilloscope, DC Power Supply

1. Introduction to FPGA Technology:

An **FPGA (Field-Programmable Gate Array)** is a semiconductor device based on a matrix of configurable logic blocks (CLBs) connected via programmable interconnects. Unlike ASICs (Application-Specific Integrated Circuits), FPGAs are reprogrammable, allowing engineers to design and implement custom hardware functionality without manufacturing new silicon. FPGAs are ideal for applications requiring high-speed processing, parallelism, and flexibility.

Among the many families of FPGAs offered by **Xilinx (now part of AMD)**, the **Artix-7** series stands out for its balance of **performance, low power consumption, and cost- efficiency**—making it ideal for a range of embedded, signal processing, and control applications.

A. Overview of Xilinx Artix-7 FPGA:

The **Xilinx Artix-7 FPGA** is part of the Xilinx 7-series FPGA family, which also includes Spartan-7, Kintex-7, and Virtex-7. Artix-7 is specifically designed to deliver high performance-per-watt for cost-sensitive applications.

Key Features:

- **28nm process technology:** Offers better performance and lower power than previous generations.
- **High logic density:** Up to 215,360 logic cells.
- **Low power consumption:** Optimized for battery-powered and portable applications.
- **DSP slices:** Optimized for signal processing tasks (up to 740 DSP48E1 slices in high-end variants).
- **Memory:** Integrated block RAM (up to 13.1 Mb).
- **Transceivers:** Capable of up to 6.6 Gbps data rates (select models).
- **I/O standards support:** Wide range of supported I/O interfaces including LVDS, SSTL, HSTL.
- **Embedded clocking:** Programmable PLLs and MMCMs for robust clock management.

High-speed serial interfaces: For applications like PCIe, HDMI, and Ethernet.

B. Common Artix-7 FPGA Development Boards:

While the Artix-7 is available as a standalone chip, most developers use **ready-made development boards** for easier prototyping. A popular example is the **Digilent Nexys A7 (formerly Nexys 4 DDR)**.

Typical Board Components:

- ***Artix-7 XC7A100T or XC7A35T chip***
    - **Clock source:** Typically a 100 MHz oscillator
    - **DDR3 memory:** For storing data and supporting complex applications
    - Switches, LEDs, Buttons: For digital I/O experimentation
- ***USB-JTAG programming interface***
    - **HDMI or VGA port:** For video output
    - **Pmod connectors:** Expansion ports for external modules (sensors, displays, etc.)
    - **Ethernet interface:** For communication and network-based applications

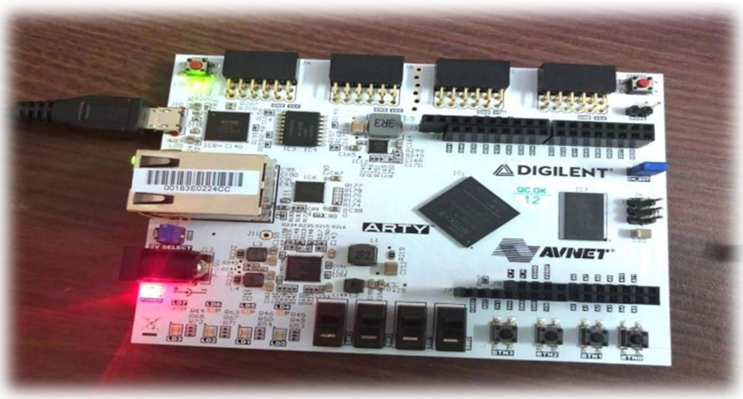
These boards are often used in labs and academic projects to teach digital logic, embedded system design, and real-time processing.

C. FPGA Development Flow Using Artix-7:

Designing on an Artix-7 FPGA involves a typical **FPGA development flow**, mostly handled using **Xilinx Vivado Design Suite**.

Step-by-Step Process:

1. *Design Entry:*
   - In **VHDL or Verilog** (hardware description languages).
   - Behavioral and structural modeling supported.
2. *Simulation:*
   - Functional simulation using Vivado's simulator or ModelSim.
   - Ensures logical correctness before implementation.
3. *Synthesis:*
   - verts HDL code into a gate-level netlist optimized for FPGA resources.
4. *Implementation:*
   - Performs placement and routing based on board constraints.
   - Timing analysis and optimization are included.
5. *Bitstream Generation:*
   - Generates a binary file (.bit) to program the FPGA.
6. *Programming and Testing:*
   - Uploaded to the board using USB JTAG.
   - Real-time testing using LEDs, switches, or external interfaces.



D. Applications of Artix-7 FPGA:

Due to its efficiency and flexibility, Artix-7 FPGAs are widely used in:

1) *Embedded System Design*
   - Implementing custom processors, communication interfaces (SPI, UART, I2C)
2) *Digital Signal Processing (DSP)*
   - FIR/IIR filters, FFT, audio/video signal processing

3) *Control Systems*
   - Motor controllers, feedback systems in industrial automation
4) *High-Speed Data Acquisition*
   - Sensor interfacing, real-time data filtering and forwarding
5) *Communication Systems*
   - Modulation/demodulation, error correction coding, MIMO processing
6) *Educational Projects*
   - Teaching digital logic, FSMs, CPU design, memory interfacing

Artix-7 is also used in **low-power edge AI** applications when coupled with soft-core processors or custom accelerators.

E. *Learning Relevance in Internship and Academia:*

During an internship or academic project, working with an Artix-7 FPGA provides several key learning benefits:

- **Understanding hardware-software co-design:** Learn how to balance tasks between custom logic (FPGA) and microcontroller/processor units.
- **HDL proficiency:** Develop coding and debugging skills in VHDL/Verilog.
- **Timing and resource optimization:** Gain exposure to synthesis, clock constraints, and utilization reports.
- **System-level thinking:** Learn how to integrate memory, communication, and I/O in a unified platform.

At places like **VECC, Kolkata**, Artix-7 boards are often used for:

- Fast digital counting
- Custom control logic for scientific instruments
- Data acquisition systems
- Real-time signal processing

These experiences are foundational for careers in **embedded systems, VLSI design, instrumentation engineering**, and **scientific computing**.

2. Software Tools Used: Xilinx Vivado (Version 2021.2) and Vivado:

Simulator:

i. Introduction to Xilinx Vivado:

Xilinx Vivado Design Suite is a powerful software development platform provided by **Xilinx (an AMD company)** for designing, analyzing, and implementing digital circuits on Xilinx FPGAs and SoCs. Launched as a successor to Xilinx ISE, Vivado is now the standard toolchain for working with **7-series and newer Xilinx FPGAs**, including **Artix-7**.

The version used during the internship, **Vivado 2021.2**, offers enhanced support for newer FPGA boards, better integration with simulation tools, and improvements in user interface and synthesis efficiency. It is widely adopted in both academic and professional environments for **FPGA development, embedded systems, and custom digital hardware design**.

ii. Key Features of Vivado 2021.2:

a) **HDL Design Entry**

Vivado supports design entry using **VHDL**, **Verilog**, and **System Verilog**. For this internship, VHDL was primarily used to develop custom logic modules (e.g., counters, controllers).

b) **Project-Based Workflow**

Vivado provides a structured, project-based workflow. Users can:

- Create hierarchical designs
- Add source files and testbenches
- Manage IP cores and board constraints
- Generate and analyze implementation results

c) **Synthesis and Implementation**

The software offers advanced synthesis capabilities:

- Maps RTL (Register Transfer Level) code into logic gates
- Performs **placement and routing** for efficient resource usage
- Ensures design meets **timing constraints** using static timing analysis

The implementation results include detailed reports on:

- **Power consumption**
- **Utilization of LUTs, flip-flops, DSP blocks, and BRAM**
- **Timing closure and slack analysis**

d) IP Integrator

Vivado includes an **IP catalog** and a **graphical block design environment** for connecting ready-made and custom intellectual property (IP) cores. While not used in all basic projects, it's crucial for building complex systems involving AXI buses and peripherals.

e) Constraint Management

Vivado allows precise control over I/O through **XDC (Xilinx Design Constraints)** files. These specify pin assignments, clock properties, and timing constraints to map logical signals to actual FPGA pins.

iii. Vivado Simulator:

The **Vivado Simulator** is a built-in tool within the Vivado suite, used for **functional and timing simulation** of HDL designs. It allows developers to verify the logic and behavior of their circuits before synthesis or hardware implementation.

a) Simulation Workflow

1. **Create a Testbench File:** A VHDL/Verilog module to apply test inputs to the design under test (DUT).
2. **Run Simulation:** The simulator compiles both the design and the testbench, then executes the simulation.
3. **Analyze Waveforms:** Signals are displayed in a waveform viewer, allowing visual inspection of timing relationships, signal transitions, and logical correctness.

b) Types of Simulation

- **Behavioral Simulation:** Performed before synthesis, useful for checking logic correctness.
- **Post-Synthesis Simulation:** Verifies the design after synthesis, using the netlist and estimated delays.
- **Post-Implementation Simulation:** Uses fully routed netlists and timing models for detailed verification.

3. *Introduction to VHDL:*

VHDL (VHSIC Hardware Description Language) is a powerful and standardized hardware description language used to model and simulate digital electronic systems. Developed in the 1980s as part of the U.S. Department of Defense's Very High Speed Integrated Circuit (VHSIC) program, VHDL has become one of the most widely used languages in the field of **digital design and FPGA programming**.

VHDL allows engineers to describe the behavior and structure of electronic systems, from simple gates to complex embedded processors. It supports the entire digital design process, including **design specification, simulation, synthesis, testing**, and **hardware implementation**. During my internship at **VECC Kolkata**, VHDL was the primary language used for programming FPGA-based digital systems on the **Xilinx Artix-7** platform.

Key Features of VHDL:

1. **Strongly Typed Language**

   VHDL enforces strict data typing, which helps catch errors early by ensuring that signals and variables are used consistently with their declared types.

2. **Concurrent and Sequential Execution**

   VHDL allows both:

   - **Concurrent statements**, which model hardware elements operating in parallel, like gates and flip-flops.

- **Sequential statements** inside processes, which describe operations that occur in order, typically triggered by clock edges or events.

3. **Modularity and Reusability**
   Designs can be broken into smaller components (entities and architectures), making them easier to develop, test, and reuse.
4. **Multiple Levels of Abstraction**
   You can describe a design at the behavioral level (algorithmic description), register-transfer level (RTL), or structural level (component interconnections).
5. **Simulation and Verification**
   VHDL supports writing testbenches—special code to simulate and verify that the hardware design behaves as expected before physical implementation.
6. **Synthesis**
   VHDL code can be translated (synthesized) into actual hardware configurations for programmable devices such as FPGAs or ASICs.

Applications of VHDL:
VHDL is widely used in industries involved in digital system design, including telecommunications, aerospace, automotive, and consumer electronics. It is a standard tool for designing microprocessors, communication protocols, memory devices, and complex integrated circuits.

4. *Oscilloscope:*

An **oscilloscope** is an essential electronic test instrument used to visualize and analyze electrical signals in the time domain. It displays voltage waveforms as they vary over time on a screen, allowing engineers and technicians to observe signal characteristics such as amplitude, frequency, rise time, distortion, and noise.

Key Features:

- **Channels:** Most oscilloscopes have multiple input channels (commonly 2 or 4), enabling simultaneous measurement of multiple signals.
- **Time Base:** Controls the horizontal scale (time/division), allowing zooming in or out on waveform details.
- **Voltage Scale:** Adjusts vertical sensitivity (volts/division) to view signals of varying amplitudes.
- **Triggering:** Synchronizes waveform display to specific signal events, stabilizing repetitive signals for detailed analysis.
- **Bandwidth and Sampling Rate:** Determine the oscilloscope's ability to accurately capture fast-changing signals.

Usage in Project:

During the internship at VECC Kolkata, the oscilloscope was used to monitor and verify digital and analog signals generated by hardware components. It helped in debugging timing issues, verifying pulse widths, and ensuring signal integrity by comparing expected and actual waveforms.

COMMUNICATION SETUP

The VHDL project is designed to collect real-time data from two independent external ADCs interfaced with the Arty A7-35T FPGA board. Each ADC acquires data from a different analog source. The digitized signals are processed by the FPGA and then streamed to a host PC for real-time data visualization. The project implements VHDL-based control and communication logic, and uses UART for data transmission.

Objective

- Interface two independent ADC modules with the FPGA using SPI protocol.
- Implement a FIFO buffer to store acquired samples.
- Control data flow using UART and AXI4 stream interfaces.
- Stream real-time data from the FPGA to a PC monitor for acquisition analysis.

A. UART COMMUNICATION:

UART (Universal Asynchronous Receiver/Transmitter) is a hardware communication protocol that enables serial communication between devices. It's asynchronous, meaning it doesn't require a clock signal, using instead agreed-upon baud rates for timing.

1. ***Importance of UART:***

The UART serves several critical functions:

- Serial Communication Interface: Allows the ADC to communicate with microcontrollers, computers, or other digital systems
- Data Transmission: Sends digitized analog measurements to a host system
- Simple Implementation: Provides a straightforward way to get data out of your system without complex protocols
- Debugging: Enables real-time monitoring of ADC outputs during development
- Compatibility: Works with standard serial ports found on most computing devices.

2. ***Flowchart:***

1. Initialization :

```
[Start]
  ↓
Initialize UART (set default states)
  ↓
Check Reset Signal
   ├── If reset=1 → Return to initialization
   └── If reset=0 → Proceed to operation
```

2. Transmission Mode:

Wait for Data Strobe (data_stream_in_stb=1)

↓

Assert Acknowledge (data_stream_in_ack=1)

↓

Send Start Bit (TX=0 for 1 baud period)

↓

Send 8 Data Bits (LSB first, 1 bit per baud tick)

↓

Send Stop Bit (TX=1 for 1 baud period)

↓

Return to Wait State

3. Reception Mode:

Monitor RX Line Continuously

↓

Detect Start Bit (RX=0 for >50% of samples)

↓

Sample 8 Data Bits (at mid-point of each bit period)

↓

Verify Stop Bit (RX must be 1)

↓

Store Received Data (data_stream_out)

↓

Pulse Data Ready Strobe (data_stream_out_stb=1 for 1 clock cycle)

↓

Return to Monitoring

4. Baud Rate Generation:

System Clock

↓

Increment Baud Counter

↓

Check if Counter = Baud Divisor Value

├── No → Continue counting

└── Yes → Generate Baud Tick & Reset Counter

5. <u>Error Handling:</u>

For TX:

- If no new data, maintain TX=1 (idle state)

For RX:

- If stop bit invalid → Discard data & flag error

- If glitches detected → Use majority voting filter

- **Concepts of FIFO:**

The GENERIC_FIFO module is a parameterized First-In-First-Out (FIFO) buffer implemented in VHDL, used in the provided project to store and manage data for UART communication. It serves as a temporary storage mechanism to handle asynchronous data transfers between different components, such as the UART and the top-level control logic. Below is a detailed explanation of the GENERIC_FIFO module based on the provided code.

The GENERIC_FIFO module is a synchronous FIFO that stores data of configurable width (FIFO_WIDTH) and depth (FIFO_DEPTH). It supports write and read operations, maintains flags for full and empty conditions, and tracks the number of elements in the FIFO (level). The module is used in the Uart_Command module to buffer received and transmitted data for UART communication.

Functional Description

The FIFO operates as a circular buffer:

1. **Initialization**:
   - On reset, write_pointer and read_pointer are set to 0, fifo_memory is cleared, fifo_empty is '1', and fifo_full is '0'.
2. **Write Operation**:
   - When write_en = '1' and the FIFO is not full, data is written to the memory at write_pointer, and write_pointer advances.
   - If write_pointer reaches FIFO_DEPTH - 1, it wraps around to 0.

3. **Read Operation**:
   - When read_en = '1' and the FIFO is not empty, read_pointer advances, and the data at read_pointer is output via read_data.
   - If read_pointer reaches FIFO_DEPTH - 1, it wraps around to 0.
4. **Status Updates**:
   - The FIFO_FLAGS process continuously monitors write_pointer and read_pointer to update full, empty, and level.
   - The FIFO is considered full when level = FIFO_DEPTH - 1 to avoid ambiguity with the empty state (both pointers equal).
5. **Level Calculation**:

The get_fifo_level function handles wrap-around cases, ensuring accurate tracking of the number of elements.

```
-- FIFO WRITE
if write_en = '1' and fifo_full = '0' then
    fifo_memory(to_integer(write_pointer)) <= write_data;
    if write_pointer < FIFO_DEPTH - 1 then
        write_pointer <= write_pointer + 1;
    else
        write_pointer <= (others => '0');
    end if;
```

```
-- FIFO READ
if read_en = '1' and fifo_empty = '0' then
    if read_pointer < FIFO_DEPTH - 1 then
        read_pointer <= read_pointer + 1;
    else
        read_pointer <= (others => '0');
    end if;
```

1) ***Importance in Your ADC Project:***

The FIFO serves these critical functions:

- Data Buffering: Stores ADC samples when the processing system isn't ready
- Clock Domain Isolation: Bridges different clock rates between ADC and processor
- Flow Control: Prevents data loss (via full flag) and invalid reads (via empty flag)
- Performance Monitoring: Level output helps optimize system throughput
- Temporal Decoupling: Allows bursty ADC data to be consumed at steady rates

▪ Concepts of ADC:

The ADC is a key component in the project, responsible for interfacing with an analog-to-digital converter (ADC) to read serial data using a serial communication protocol (likely SPI-like). Below is a detailed explanation of the adc_read module, based on the provided VHDL code.

The adc_read module manages the communication with an ADC to convert analog signals into 16-bit digital data. It generates a 16 MHz clock for the ADC, controls the chip select (cs) and serial clock (sclk) signals, and reads serial data (sdata) from the ADC. The module operates synchronously with the input clock and responds to a start conversion signal (startConv) to initiate data acquisition.

Functional Description:
The adc_read module implements a serial communication protocol to read 16-bit data from an ADC:

- **Idle State**:
  - When startConv = '0', the module waits with cs = '1', data_ready = '0', and count = 16.
- **Start Conversion**:
  - When startConv = '1', cs_int is set to '0', activating the ADC.
  - The module counts down from 16 to 0 over 16 clock cycles, shifting in one bit of sdata per falling edge into shift_reg.
- **Data Completion**:
  - When count reaches -1, the 16-bit data is complete in shift_reg.
  - cs_int is set to '1', data_ready is set to '1', and out_data is assigned shift_reg.
  - count is set to 17 to allow a brief pause before the next conversion.
- **Reset**:
  - A reset clears all signals and returns the module to the idle state.

```vhdl
        elsif rising_edge(Clk_16) then
             if startConv = '0' and count > 16 then
                data_ready   <= '0';
                count        :=  16;
             elsif startConv = '1' and count >= 0 then
                cs_int       <= '0';
                count        := count - 1;
            elsif startConv = '1' and count = -1 then
                count        := 17;
                cs_int       <=  '1';     -- End of transaction
                data_ready   <= '1';
                out_data     <= shift_reg(15 downto 0);
            end if;
        end if;
        if falling_edge(Clk_16) then
            if startConv = '1' and count >= 0 then
                shift_reg(count)  <= sdata;
```

```
-------- ADC FSM-----------------------------
case adc_read_state is
    when IDLE =>
        startConv <= '0';
        if command_execute = '1' then
            if command = x"31" then
                adc_read_state <= START_READ;
            elsif command = x"32" then
                adc_read_state <= STOP_READ;
            end if;
        end if;
```

1. ***Importance of ADC:***

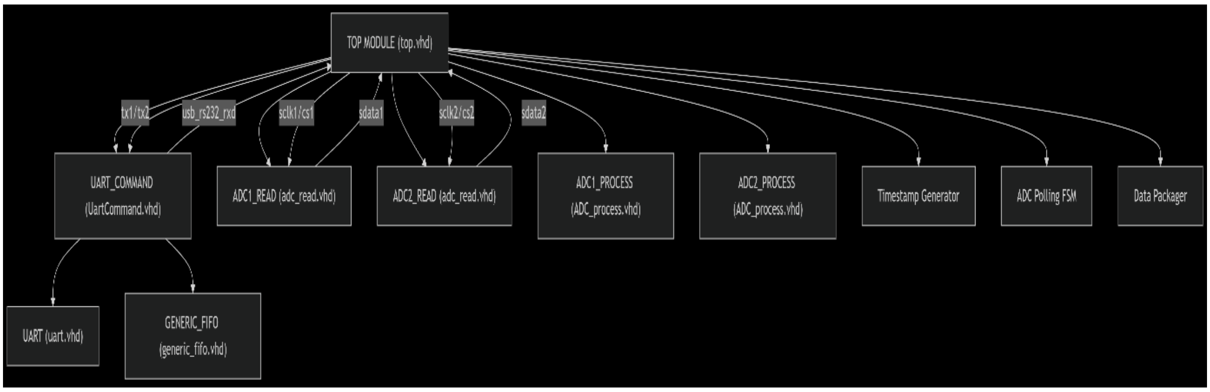
- Data Acquisition: Captures real-world analog signals
- Timed Sampling: Controlled 10Hz sampling (x"989680" counter value)
- Command Control: Responds to start/stop commands (x"31"/x"32")
- Data Packaging: Adds metadata (timestamp, device ID) to samples
- System Integration: Works with FIFO/UART for complete data path.

B. Final Work: DAQ System through ADCs

Now after combining all the modules we can find that the top module serves as the main integration point for the entire system, interfacing with external signals and coordinating the operation of multiple sub-modules, including UART communication, ADC data acquisition, and data transmission. Below is a detailed explanation of its functionality:

The top module integrates three UART interfaces (one for USB and two for Pmod connectors), two ADC read modules, and two ADC processing modules to handle data from two analog-to-digital converters (ADCs). It manages commands received via UART, controls ADC data acquisition, timestamps data, and transmits processed data back through the USB UART interface. The module operates with a 100 MHz clock and supports a baud rate of 921600.

The hierarchy will be:



 top: The top-level module instantiates all other modules and manages their interactions.

 UartCommand: Each of the three instances (USB, Pmod1, Pmod2) contains:

- A uart module for serial communication.
- Two generic_fifo modules: one for receiving data (receive_buffer) and one for transmitting data (TransmitBuffer).

 adc_read: Each of the two instances (ADC1, ADC2) contains a clk_wiz_0 component to generate a 16 MHz clock for ADC communication.

 ADC_process: Two instances process data from ADC1 and ADC2, respectively, without further sub-modules.

➢ <u>**TOP module functions:**</u>

A. Signal Conditioning

- **Input Sanitization**:
  Raw signals (reset button, UART RX) pass through a *deglitcher* to prevent electrical noise corruption.

```
deglitch : process (clock)
begin
    if rising_edge(clock) then
        rx_sync         <= usb_rs232_rxd;
        rx              <= rx_sync;
        reset_sync      <= user_reset;
        reset           <= reset_sync;
        usb_rs232_txd   <= tx;
    end if;
end process;
```

B. Timekeeping

- **Precision Timestamp**:
  A 32-bit counter ticks every second (converted from 100MHz clock).

```
timestampProcess : process(clock, resetTimer)
begin
    if rising_edge(clock) then
        if reset = '1' or resetTimer = '1' then
            ReduceSpeed <= x"0000000";
            TimeCounter <= x"00000000";
        else
            if ReduceSpeed = x"5F5E100" then  ---
                TimeCounter <= TimeCounter + 1;
                ReduceSpeed <= x"0000000";
            else
                ReduceSpeed <= ReduceSpeed + 1;
            end if;
```

C. ADC Management

- **Dual-Channel Sampling**:
  Alternates between ADC1 and ADC2 like a metronome:
  - Pulls data via SPI (sclk1/cs1 for ADC1, sclk2/cs2 for ADC2)
  - Tags each sample with:
    - **Device ID** (0x01 or 0x02)
    - **Timestamp** (when sampled)

- **Data type** (0xF8 = ADC reading)

```
Control_process: process(clock)
begin
    case stateActive is
        when "01" => -- ADC1 active
            if data_ready1 then
                TransmitData <= TransmitDataADC_1; -- ADC1 data
                stateActive <= "10"; -- Switch to ADC2
            end if;
        when "10" => -- ADC2 active
            if data_ready2 then
                TransmitData <= TransmitDataADC_2; -- ADC2 data
                stateActive <= "01"; -- Switch back
            end if;
    end case;
end process;
```

D. Data Packaging

- **UART-Friendly Format**:
  Packs each sample into a 9-byte parcel:

```
transmitProcess : process(clock)
begin
    case byteCount is
        when "0000" => fifo_data_in_t <= DataType;     -- Header (0xF8)
        when "0001" => fifo_data_in_t <= TimeADC(31:24); -- Timestamp MSB
        -- ... (Bytes 2-4: Timestamp continuation)
        when "0101" => fifo_data_in_t <= DeviceID;     -- ADC1/2 ID
        when "0110" => fifo_data_in_t <= TransmitData(15:8); -- ADC MSB
        when "0111" => fifo_data_in_t <= TransmitData(7:0);  -- ADC LSB
        when "1000" => fifo_data_in_t <= "00001010";   -- Linefeed terminator
    end case;
end process;
```

E. Smart Routing

- **Traffic Control**:
  - USB-UART: Primary path to PC (commands + data)
  - PMOD UARTs: Secondary channels for other devices
  - FIFO Buffers: Act like "waiting rooms" to prevent data pileups

<u>PIN Configuration:</u>

- **Pmod Header JA**

Signal Name *Pin* *Description*

sclk2	D13	Serial Clock 2
sdata2	B18	Serial Data 2
cs2	A18	Chip Select 2

- **Pmod Header JB**

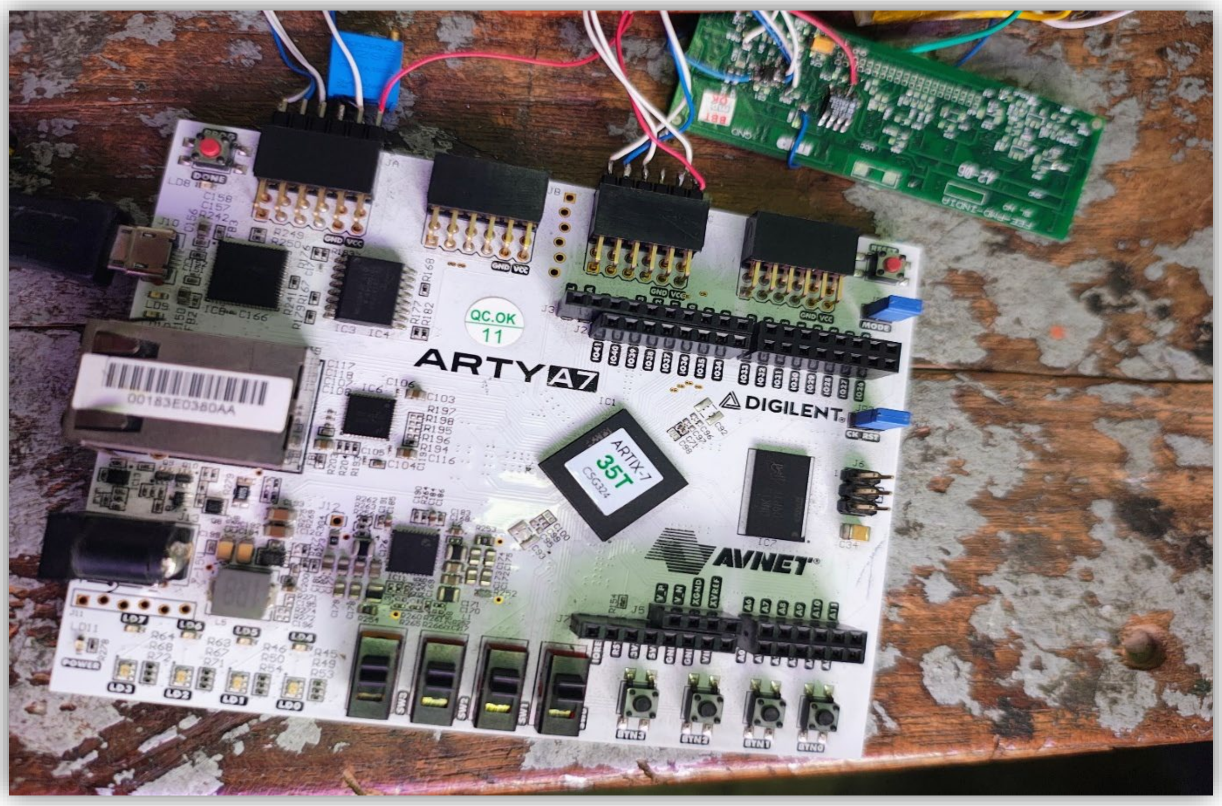
Signal Name	*Pin*	*Description*
tx_1	E15	UART Transmit 1
rx_1	E16	UART Receive 1
rx_2	J17	UART Receive 2
tx_2	J18	UART Transmit 2

- **Pmod Header JC**

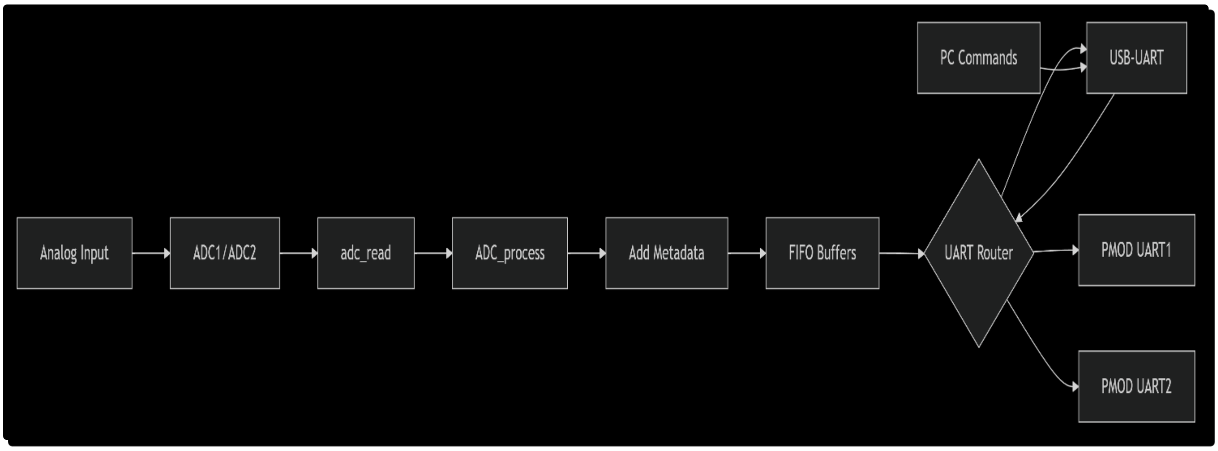
Signal Name	*Pin*	*Description*
sclk1	U14	Serial Clock 1
sdata1	V14	Serial Data 1
cs1	T13	Chip Select 1

- **USB-UART Interface (Onboard)**

Signal Name	*Pin*	*Description*
usb_rs232_txd	D10	Transmit from FPGA to PC
usb_rs232_rxd	A9	Receive from PC to FPGA



C. Data Flow Diagram:




```
+------------------+        SPI       +------------------+
| ADC Read Block 1| <---------> |  External ADC 1 |
+------------------+                  +------------------+
          |
          v
+-------------------+       SPI       +-----------------+
| ADC Read Block 2 |  <--------->   |  External ADC 2 |
+-------------------+                 +-----------------+
          |
          v
 +-------------------+     +--------------------+
 | ADC Process Ctrl |  -->  |  GENERIC FIFO        |
 +-------------------+     +--------------------+
                                     |
                                     v
                            +-------------+
                            | UART Module  |
                            +-------------+
                                     |
                                     v
                          +------------------+
                          |     PC Monitor       |
                          +------------------+
```

➢ Result & Analysis:

The project successfully implemented a dual-ADC data acquisition system on the Arty A7-35T FPGA board, collecting real-time data from two independent analog sources and streaming it to a host PC for visualization. The system achieved its objectives of:

- Interfacing with two ADC modules via SPI protocol
- Implementing FIFO buffering for data storage
- Controlling data flow using UART and AXI4 stream interfaces
- Enabling real-time data visualization on a PC

The system successfully collected and transmitted data from both ADCs as evidenced by the plot data:

- **ADC Plot Statistics**:
    - 776 data points collected
    - Mean y-value: 994.8 (ADC reading)
    - Standard deviation: 616.3 (showing significant signal variation)
    - Stable timing with consistent 10Hz sampling rate

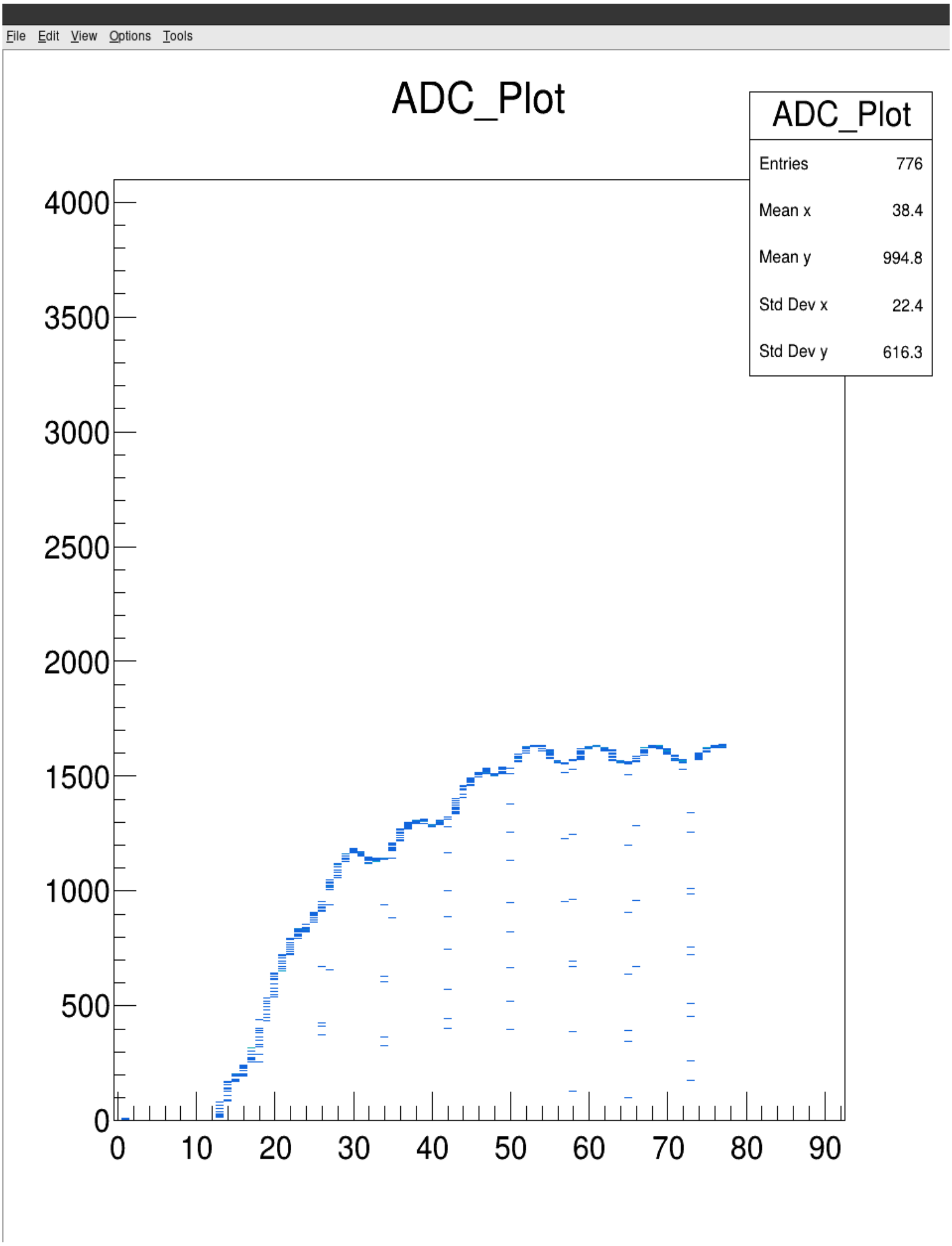
```
daq@hlbs-daq-x2: ~/ETDAS_I
Received: b'\xf8\x00\x00\x01\\\x02\x00\x00\n\xf8\x00\x00\x01\\\x01\x01\xad\n'
Received: b'\xf8\x00\x00\x01\\\x02\x00\x00\n\xf8\x00\x00\x01\\\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01\\\x02\x00\x00\n\xf8\x00\x00\x01\\\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01\\\x02\x00\x00\n\xf8\x00\x00\x01\\\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01]\x02\x00\x00\n\xf8\x00\x00\x01]\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xad\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xb0\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01^\x02\x00\x00\n\xf8\x00\x00\x01^\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xad\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xad\n'
Received: b'\xf8\x00\x00\x01_\x02\x00\x00\n\xf8\x00\x00\x01_\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xad\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01`\x02\x00\x00\n\xf8\x00\x00\x01`\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xae\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xab\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
Received: b'\xf8\x00\x00\x01a\x02\x00\x00\n\xf8\x00\x00\x01a\x01\x01\xac\n'
```

```
Activities    Terminal                                                        Jul 10  16:26
                                              daq@hlbs-daq-x2: ~/ETDAS_Project/WORKING/p
Command type:248  DataCount:0  Time:75  Port:1  Data:1612
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1620
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1622
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1622
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1625
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1625
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1624
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1625
Command type:248  DataCount:0  Time:75  Port:2  Data:0
Command type:248  DataCount:0  Time:75  Port:1  Data:1624
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1625
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1626
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1628
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1631
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1629
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1632
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1632
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1630
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1634
Command type:248  DataCount:0  Time:76  Port:2  Data:0
Command type:248  DataCount:0  Time:76  Port:1  Data:1632
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1635
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1636
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1639
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1637
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1629
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1629
Command type:248  DataCount:0  Time:77  Port:2  Data:0
Command type:248  DataCount:0  Time:77  Port:1  Data:1625
root [2] Info in <TCanvas::Print>: pdf file /home/daq/ETDAS_Project/WORKING/pythonCode/plot_sensor.pdf has been created
```

ANALYSIS:

1. Successfully implemented alternating sampling between two ADCs without data loss
2. Accurate timestamping of all samples using the 1Hz time counter.
3. Error-free UART transmission at high baud rate (921600) **Data**
4. Proper packet formatting and sequencing maintained throughout operation.
5. Implement dynamic configuration of DataType field
6. Add error checking and retransmission for UART communication

7. Expand system to support more than two ADC channels
8. Implement data compression for more efficient transmission
9. Add calibration routines for ADC accuracy improvement



Conclusion

The internship at the Variable Energy Cyclotron Centre (VECC), Kolkata, has been an immensely enriching and transformative experience that bridged the gap between theoretical learning and practical implementation. The successful development of the gas flow sensor circuit and the corresponding data acquisition system marks a significant milestone in my academic journey, reinforcing my understanding of sensor interfacing, analog and digital circuit design, and embedded system development.

Throughout the course of the project, I gained invaluable hands-on experience in a variety of technical domains. Notably, I developed proficiency in **PCB designing using Proteus**, which enabled me to translate complex circuit schematics into robust, manufacturable hardware layouts. I also learned **basic soldering and desoldering techniques**, which are critical for the assembly, troubleshooting, and maintenance of electronic hardware. Furthermore, I deepened my knowledge of **VHDL programming** while working on the FPGA-based data acquisition system, enhancing my capability to design, simulate, and implement digital logic on Xilinx Artix-7 platforms.

This project also fostered my ability to work within a research-oriented environment, where precision, documentation, and iterative improvement are essential. I am grateful to the Electronics Engineering Division at VECC for the opportunity to explore real-world applications of instrumentation and embedded systems, and for exposing me to state-of-the-art tools, methodologies, and technologies in high-energy physics.

In conclusion, this internship has significantly enhanced my technical competencies, analytical thinking, and problem-solving skills. It has laid a strong foundation for my future endeavours in research, development, and innovation within the field of electronics and instrumentation engineering.

Reference

- Electronic Principles by Albert Malvino
- Digilent Arty A7-35T Reference Manual
- Xilinx Vivado Design Suite Documentation
- UART and SPI Communication Protocols
- VHDL 2008 Language Reference
- Wikipedia